17.3 身份系统

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~450,000 tokens，输出 ~36,000 tokens（本章合计）

OpenClaw 的每个 Agent 不只是一个"无名的助手"——它可以有名字、有 emoji 标识、有自定义头像、有独特的回复风格。这套**身份系统**让用户能够给 Agent 一个具体的形象，同时在多 Agent 场景中帮助区分不同 Agent 的消息。

身份系统由三层组成：配置层（`identity.ts`）、文件层（`identity-file.ts`）、和头像层（`identity-avatar.ts`）。

17.3.1 AI 助手身份（`src/agents/identity.ts`）

IdentityConfig：身份配置类型

Agent 身份从 OpenClaw 配置文件中读取。每个 Agent 可以在 `agents.[id].identity` 下定义身份信息：

```yaml
# openclaw.yaml 示例
agents:
  agents:
    - id: default
      identity:
        name: "Aria"
        emoji: "🦊"
```

`identity.ts` 提供了一组解析函数，从配置中提取身份信息：

```
// src/agents/identity.ts
const DEFAULT_ACK_REACTION = "👀";

export function resolveAgentIdentity(
  cfg: OpenClawConfig,
  agentId: string,
): IdentityConfig | undefined {
  return resolveAgentConfig(cfg, agentId)?.identity;
}
```

`resolveAgentConfig` 来自 `agent-scope.ts`，它通过 Agent ID 在配置的 agents 列表中查找对应条目，并返回标准化后的配置对象。

确认反应（Ack Reaction）

用户发送消息时，OpenClaw 会先发送一个"确认"表情反应（类似于 iMessage 的"已读"），告知用户消息已收到。默认是 👀，但可以自定义：

```
export function resolveAckReaction(
  cfg: OpenClawConfig,
  agentId: string
): string {
  // 优先级 1：全局配置 messages.ackReaction
  const configured = cfg.messages?.ackReaction;
  if (configured !== undefined) return configured.trim();

  // 优先级 2：Agent 身份 emoji
  const emoji = resolveAgentIdentity(cfg, agentId)?.emoji?.trim();
  return emoji || DEFAULT_ACK_REACTION;
}
```

这意味着如果 Agent 有自定义 emoji（如 🦊），确认反应就会变成那个 emoji 而不是默认的 👀。

消息前缀系统

在多 Agent 场景或群聊中，需要区分哪条消息来自哪个 Agent。OpenClaw 通过**消息前缀**实现：

```
// 生成身份名前缀，格式：[Name]
export function resolveIdentityNamePrefix(
  cfg: OpenClawConfig,
  agentId: string,
): string | undefined {
  const name = resolveAgentIdentity(cfg, agentId)?.name?.trim();
  if (!name) return undefined;
  return `[${name}]`;  // 例如 "[Aria]"
}
```

消息前缀的解析有一套复杂的优先级逻辑：

```
export function resolveMessagePrefix(
  cfg: OpenClawConfig,
  agentId: string,
  opts?: { configured?: string; hasAllowFrom?: boolean; fallback?: string },
): string {
  // 优先级 1：显式配置
  const configured = opts?.configured ?? cfg.messages?.messagePrefix;
  if (configured !== undefined) return configured;

  // 优先级 2：有 allowFrom 的通道不需要前缀
  if (opts?.hasAllowFrom === true) return "";

  // 优先级 3：身份名前缀 → 回退默认 "[openclaw]"
  return resolveIdentityNamePrefix(cfg, agentId) ?? opts?.fallback ?? "[openclaw]";
}
```

> **衍生解释——allowFrom 与消息前缀的关系**
>
> 在某些通道（如 Signal、Telegram）中，OpenClaw 使用专用的号码/bot 账号发送消息。如果配置了 `allowFrom` （即指定哪些号码的消息被视为来自 OpenClaw），通道本身已经能区分消息来源，因此不需要额外的 `[名字]` 前缀。

回复前缀（Response Prefix）

与消息前缀不同，**回复前缀**用于 Agent 在回复用户消息时的标识。它支持四级优先配置：

```typescript
export function resolveResponsePrefix(
  cfg: OpenClawConfig,
  agentId: string,
  opts?: { channel?: string; accountId?: string },
): string | undefined {
  // L1: 通道账号级别（最高优先级）
  if (opts?.channel && opts?.accountId) {
    const accountPrefix =
      channelCfg?.accounts?.[opts.accountId]?.responsePrefix;
    if (accountPrefix !== undefined) {
      if (accountPrefix === "auto")
        return resolveIdentityNamePrefix(cfg, agentId);
      return accountPrefix;
    }
  }

  // L2: 通道级别
  if (opts?.channel) {
    const channelPrefix = channelCfg?.responsePrefix;
    if (channelPrefix !== undefined) {
      if (channelPrefix === "auto")
        return resolveIdentityNamePrefix(cfg, agentId);
      return channelPrefix;
    }
  }

  // L4: 全局级别
  const configured = cfg.messages?.responsePrefix;
  if (configured !== undefined) {
    if (configured === "auto")
      return resolveIdentityNamePrefix(cfg, agentId);
    return configured;
  }
  return undefined;
}
```

`"auto"` 是一个特殊值，表示"自动使用 Agent 身份名"。这样配置既简洁又灵活：

```yaml
# 全局设置自动前缀
messages:
  responsePrefix: "auto"   # → 自动变成 "[Aria]"

# 或者 Telegram 通道使用自定义前缀
channels:
  telegram:
    responsePrefix: "🤖 "
```

拟人化延迟（Human Delay）

身份系统还包含一个有趣的功能——**拟人化延迟**。它让 Agent 的回复不再"秒回"，而是模拟人类的打字延迟：

```typescript
export function resolveHumanDelayConfig(
  cfg: OpenClawConfig,
  agentId: string,
): HumanDelayConfig | undefined {
  const defaults = cfg.agents?.defaults?.humanDelay;
  const overrides = resolveAgentConfig(cfg, agentId)?.humanDelay;
  if (!defaults && !overrides) return undefined;
  return {
    mode: overrides?.mode ?? defaults?.mode,
    minMs: overrides?.minMs ?? defaults?.minMs,
    maxMs: overrides?.maxMs ?? defaults?.maxMs,
  };
}
```

配置示例：

```yaml
agents:
  defaults:
    humanDelay:
      mode: "typing"    # 显示"正在输入"状态
      minMs: 500        # 最短延迟 500ms
      maxMs: 3000       # 最长延迟 3s
```

17.3.2 身份文件（`src/agents/identity-file.ts`）

除了配置文件之外，Agent 还可以通过工作区中的 `IDENTITY.md` 文件定义身份。这提供了一种更加用户友好的方式——用户直接编辑 Markdown 文件即可自定义 Agent 形象。

AgentIdentityFile 类型

```
// src/agents/identity-file.ts
export type AgentIdentityFile = {
  name?: string;     // Agent 名字
  emoji?: string;    // 代表 emoji
  theme?: string;    // 主题色/风格
  creature?: string; // 角色设定（"AI? robot? ghost in the machine?"）
  vibe?: string;     // 语调（"sharp? warm? chaotic? calm?"）
  avatar?: string;   // 头像路径或 URL
};
```

IDENTITY.md 解析

`parseIdentityMarkdown` 从 Markdown 格式中提取键值对：

```typescript
export function parseIdentityMarkdown(content: string): AgentIdentityFile {
  const identity: AgentIdentityFile = {};
  const lines = content.split(/\r?\n/);

  for (const line of lines) {
    // 清理列表标记：移除开头的 "- "
    const cleaned = line.trim().replace(/^\s*-\s*/, "");
    const colonIndex = cleaned.indexOf(":");
    if (colonIndex === -1) continue;

    // 提取标签和值，移除 Markdown 加粗/斜体标记
    const label = cleaned.slice(0, colonIndex)
      .replace(/[*_]/g, "").trim().toLowerCase();
    const value = cleaned.slice(colonIndex + 1)
      .replace(/^[*_]+|[*_]+$/g, "").trim();

    if (!value) continue;
    if (isIdentityPlaceholder(value)) continue; // 跳过占位符

    if (label === "name") identity.name = value;
    if (label === "emoji") identity.emoji = value;
    if (label === "creature") identity.creature = value;
    if (label === "vibe") identity.vibe = value;
    if (label === "theme") identity.theme = value;
    if (label === "avatar") identity.avatar = value;
  }
  return identity;
}
```

一个典型的 IDENTITY.md 文件：

```markdown
# Identity

- **Name**: Aria
- **Emoji**: 🦊
- **Creature**: A curious fox spirit
- **Vibe**: Warm but precise
- **Theme**: sunset
- **Avatar**: avatar.png
```

占位符检测

初始模板中的占位符文本需要被识别并忽略，避免将"pick something you like"当作真实的身份值：

```
const IDENTITY_PLACEHOLDER_VALUES = new Set([
  "pick something you like",
  "ai? robot? familiar? ghost in the machine? something weirder?",
  "how do you come across? sharp? warm? chaotic? calm?",
  "your signature - pick one that feels right",
  "workspace-relative path, http(s) url, or data uri",
]);

function isIdentityPlaceholder(value: string): boolean {
  const normalized = normalizeIdentityValue(value);
  return IDENTITY_PLACEHOLDER_VALUES.has(normalized);
}
```

`normalizeIdentityValue` 会清理 Markdown 格式标记、括号、Unicode 短横线等，确保即使占位符文本被用户稍微修改格式后仍能被正确识别：

```
function normalizeIdentityValue(value: string): string {
  let normalized = value.trim();
  normalized = normalized.replace(/^[*_]+|[*_]+$/g, "").trim();
  if (normalized.startsWith("(") && normalized.endsWith(")")) {
    normalized = normalized.slice(1, -1).trim();
  }
  normalized = normalized.replace(/[\u2013\u2014]/g, "-"); // em/en dash → hyphen
  normalized = normalized.replace(/\s+/g, " ").toLowerCase();
  return normalized;
}
```

身份有效性检测

`identityHasValues` 检查解析结果是否包含至少一个有意义的值：

```
export function identityHasValues(identity: AgentIdentityFile): boolean {
  return Boolean(
    identity.name || identity.emoji || identity.theme ||
    identity.creature || identity.vibe || identity.avatar
  );
}
```

从工作区加载身份

```
export function loadAgentIdentityFromWorkspace(
  workspace: string
): AgentIdentityFile | null {
  const identityPath = path.join(workspace, DEFAULT_IDENTITY_FILENAME);
  return loadIdentityFromFile(identityPath);
}

export function loadIdentityFromFile(
  identityPath: string
): AgentIdentityFile | null {
  try {
    const content = fs.readFileSync(identityPath, "utf-8");
    const parsed = parseIdentityMarkdown(content);
    if (!identityHasValues(parsed)) return null;
    return parsed;
  } catch {
    return null; // 文件不存在或读取失败
  }
}
```

这里使用了**同步读取**（`readFileSync`）。这是有意为之——身份加载发生在 Agent 启动的关键路径上，文件通常很小（<1KB），同步读取避免了异步流程的复杂性。

17.3.3 身份头像（`src/agents/identity-avatar.ts`）

头像让 Agent 在消息通道（如 Telegram、Signal）中拥有可视化的"面孔"。`identity-avatar.ts` 负责解析和验证头像来源。

头像解析结果类型

```
// src/agents/identity-avatar.ts
export type AgentAvatarResolution =
  | { kind: "none"; reason: string }      // 无头像
  | { kind: "local"; filePath: string }    // 本地文件
  | { kind: "remote"; url: string }        // 远程 URL
  | { kind: "data"; url: string };         // Data URI
```

这个联合类型涵盖了头像的四种可能状态，使用 TypeScript 的**可辨识联合**（Discriminated Union）模式。

> **衍生解释——可辨识联合（Discriminated Union）**
>
> 可辨识联合是 TypeScript 中一种强大的类型模式。通过一个公共字段（这里是 `kind` ）来区分不同的类型分支。编译器可以根据 `kind` 的值自动推断其他字段的类型，实现类型安全的模式匹配：
>
> ```
> function handleAvatar(avatar: AgentAvatarResolution) {
>  switch (avatar.kind) {
>  case "local":
>  // TypeScript 知道这里有 avatar.filePath
>  fs.readFileSync(avatar.filePath);
>  break;
>  case "remote":
>  // TypeScript 知道这里有 avatar.url
>  fetch(avatar.url);
>  break;
>  }
> }
> ```

头像来源解析

头像值可能来自两个地方：配置文件或 IDENTITY.md 文件：

```
function resolveAvatarSource(
  cfg: OpenClawConfig,
  agentId: string
): string | null {
  // 优先级 1：配置文件中的 identity.avatar
  const fromConfig = normalizeAvatarValue(
    resolveAgentIdentity(cfg, agentId)?.avatar
  );
  if (fromConfig) return fromConfig;

  // 优先级 2：IDENTITY.md 文件中的 avatar 字段
  const workspace = resolveAgentWorkspaceDir(cfg, agentId);
  const fromIdentity = normalizeAvatarValue(
    loadAgentIdentityFromWorkspace(workspace)?.avatar
  );
  return fromIdentity;
}
```

头像类型判断

```
function isRemoteAvatar(value: string): boolean {
  const lower = value.toLowerCase();
  return lower.startsWith("http://") || lower.startsWith("https://");
}

function isDataAvatar(value: string): boolean {
  return value.toLowerCase().startsWith("data:");
}
```

本地头像路径解析与安全检查

对于本地文件路径，OpenClaw 执行严格的安全验证：

```
const ALLOWED_AVATAR_EXTS = new Set([
  ".png", ".jpg", ".jpeg", ".gif", ".webp", ".svg"
]);

function resolveLocalAvatarPath(params: {
  raw: string;
  workspaceDir: string;
}): { ok: true; filePath: string } | { ok: false; reason: string } {
  const workspaceRoot = resolveExistingPath(params.workspaceDir);

  // 解析相对路径或绝对路径
  const resolved = raw.startsWith("~") || path.isAbsolute(raw)
    ? resolveUserPath(raw)
    : path.resolve(workspaceRoot, raw);
  const realPath = resolveExistingPath(resolved);

  // 安全检查 1：必须在工作区目录内
  if (!isPathWithin(workspaceRoot, realPath)) {
    return { ok: false, reason: "outside_workspace" };
  }

  // 安全检查 2：必须是允许的图片扩展名
  const ext = path.extname(realPath).toLowerCase();
  if (!ALLOWED_AVATAR_EXTS.has(ext)) {
    return { ok: false, reason: "unsupported_extension" };
  }

  // 安全检查 3：文件必须存在且是常规文件
  try {
    if (!fs.statSync(realPath).isFile()) {
      return { ok: false, reason: "missing" };
    }
  } catch {
    return { ok: false, reason: "missing" };
  }

  return { ok: true, filePath: realPath };
}
```

安全检查包括三个维度：

1. **路径遍历防护**：通过 `isPathWithin` 确保解析后的真实路径（经过 `realpath` 解析符号链接后）仍在工作区目录内。这防止了 `../../etc/passwd` 之类的路径遍历攻击。
2. **扩展名白名单**：只允许常见的图片格式，阻止上传可执行文件或其他危险格式。
3. **文件存在性**：确保引用的文件确实存在。

```
function isPathWithin(root: string, target: string): boolean {
  const relative = path.relative(root, target);
  if (!relative) return true;
  return !relative.startsWith("..") && !path.isAbsolute(relative);
}
```

完整的头像解析流程

`resolveAgentAvatar` 将所有步骤串联起来：

```
export function resolveAgentAvatar(
  cfg: OpenClawConfig,
  agentId: string
): AgentAvatarResolution {
  const source = resolveAvatarSource(cfg, agentId);
  if (!source) {
    return { kind: "none", reason: "missing" };
  }

  if (isRemoteAvatar(source)) {
    return { kind: "remote", url: source };
  }

  if (isDataAvatar(source)) {
    return { kind: "data", url: source };
  }

  // 本地路径 → 安全验证
  const workspaceDir = resolveAgentWorkspaceDir(cfg, agentId);
  const resolved = resolveLocalAvatarPath({ raw: source, workspaceDir });
  if (!resolved.ok) {
    return { kind: "none", reason: resolved.reason };
  }

  return { kind: "local", filePath: resolved.filePath };
}
```

决策树：

```
avatar 值
  │
  ├─ 空/未设置      → { kind: "none", reason: "missing" }
  ├─ http(s)://     → { kind: "remote", url }
  ├─ data:          → { kind: "data", url }
  └─ 其他           → 视为本地路径
       │
       ├─ 路径在工作区外     → { kind: "none", reason: "outside_workspace" }
       ├─ 扩展名不允许       → { kind: "none", reason: "unsupported_extension" }
       ├─ 文件不存在         → { kind: "none", reason: "missing" }
       └─ 验证通过           → { kind: "local", filePath }
```

17.3.4 通道前缀身份

在多通道场景中，不同的消息通道可能需要不同的身份表现。OpenClaw 通过 `resolveEffectiveMessagesConfig` 统一处理消息前缀和回复前缀：

```ts
// src/agents/identity.ts
export function resolveEffectiveMessagesConfig(
  cfg: OpenClawConfig,
  agentId: string,
  opts?: {
    hasAllowFrom?: boolean;
    fallbackMessagePrefix?: string;
    channel?: string;
    accountId?: string;
  },
): { messagePrefix: string; responsePrefix?: string } {
  return {
    messagePrefix: resolveMessagePrefix(cfg, agentId, {
      hasAllowFrom: opts?.hasAllowFrom,
      fallback: opts?.fallbackMessagePrefix,
    }),
    responsePrefix: resolveResponsePrefix(cfg, agentId, {
      channel: opts?.channel,
      accountId: opts?.accountId,
    }),
  };
}
```

四级前缀优先级

回复前缀的配置支持四个层级（从高到低）：

```
L1: channels.{channel}.accounts.{accountId}.responsePrefix
L2: channels.{channel}.responsePrefix
L3: (跳过——预留给未来的 Agent 级别配置)
L4: messages.responsePrefix
```

配置示例：

```yaml
# 全局默认：不加前缀
messages:
  responsePrefix: ""

# Telegram 通道：使用 Agent 身份名
channels:
  telegram:
    responsePrefix: "auto"

# Telegram 特定账号：使用自定义前缀
channels:
  telegram:
    accounts:
      bot_123456:
        responsePrefix: "🤖 Aria: "
```

当值为 `"auto"` 时，系统自动使用 `resolveIdentityNamePrefix` 生成 `[Name]` 格式的前缀。这让配置既简洁又灵活——用户只需在 Agent 身份中设置一次 `name`，所有通道都可以通过 `"auto"` 引用。

身份信息在通道中的使用

当 OpenClaw 通过 Telegram、Signal 等通道发送消息时，身份系统的各个组件会协同工作：

```
用户消息到达
    │
    ├─ resolveAckReaction → 发送 emoji 确认反应（如 🦊）
    │
    ├─ Agent 处理消息...
    │
    └─ 发送回复
         │
         ├─ resolveEffectiveMessagesConfig
         │    ├─ messagePrefix: "[Aria]"（主动消息）
         │    └─ responsePrefix: ""（回复消息，自动路由）
         │
         └─ resolveAgentAvatar → 设置通道中的头像
```

本节小结

1. **身份系统**分为三层：配置层（`identity.ts`）处理优先级解析，文件层（`identity-file.ts`）解析 IDENTITY.md Markdown，头像层（`identity-avatar.ts`）处理多种头像来源。
2. **消息前缀**支持四级配置优先级（账号 > 通道 > 全局），`"auto"` 值自动引用 Agent 身份名，实现一处定义、多处引用。
3. **头像安全验证**包含路径遍历防护、扩展名白名单和文件存在性检查三重保障。
4. **占位符检测**通过标准化文本比较，确保模板中的引导性文本不会被误认为有效身份值。
5. **拟人化延迟**让 Agent 的回复行为更接近真人，可按 Agent 和全局两级配置。
6. 整个身份系统的设计理念是**配置优先、文件回退**——配置文件中的设置始终覆盖 IDENTITY.md 文件中的设置。